

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования
«Северный (Арктический) федеральный университет имени М.В. Ломоносова»

Высшая школа информационных технологий и автоматизированных систем
(наименование высшей школы / филиала / института / колледжа)

ОТЧЕТ
о лабораторном практикуме

По дисциплине DevTestOps

На тему Автоматизация тестирования ПО на python

Выполнил обучающийся:

Груздев Николай Александрович
(Ф.И.О.)

Направление подготовки / специальность:

09.03.01 Информатика и вычислительная
техника
(код и наименование)

Курс: 4

Группа: 151220

Руководитель: Тарасов А.П., старший
преподаватель кафедры информационных
систем и информационной безопасности САФУ
(Ф.И.О. руководителя, должность / уч. степень / звание)

Отметка о зачете

(отметка прописью)

(дата)

Руководитель

(подпись руководителя)

А.П. Тарасов
(инициалы, фамилия)

Архангельск 2025

ЛИСТ ДЛЯ ЗАМЕЧАНИЙ

ЦЕЛЬ РАБОТЫ

Получить практические навыки по автоматизации тестирования приложения на python.

1 РЕШЕНИЕ ПОСТАВЛЕННОЙ ЗАДАЧИ

Откроем веб-интерфейс GitLab и создадим новый проект для приложения «Crystal» (рис. 1).

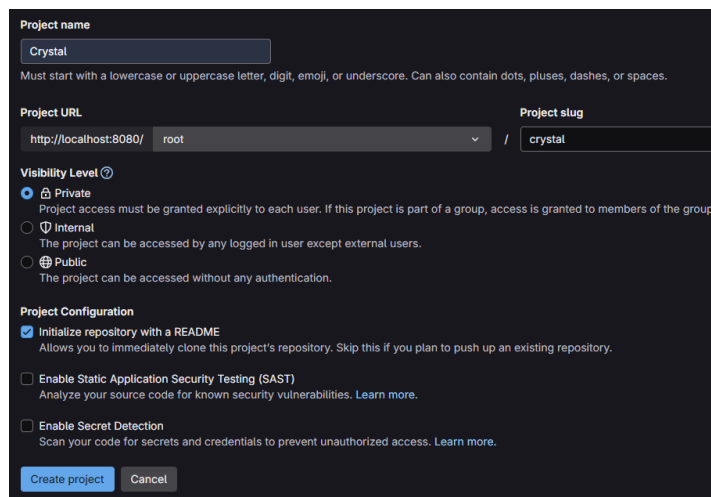


Рисунок 1 – Создание проекта в GitLab

В настройках репозитория разрешим прямые push-запросы в ветку main (рис. 2).

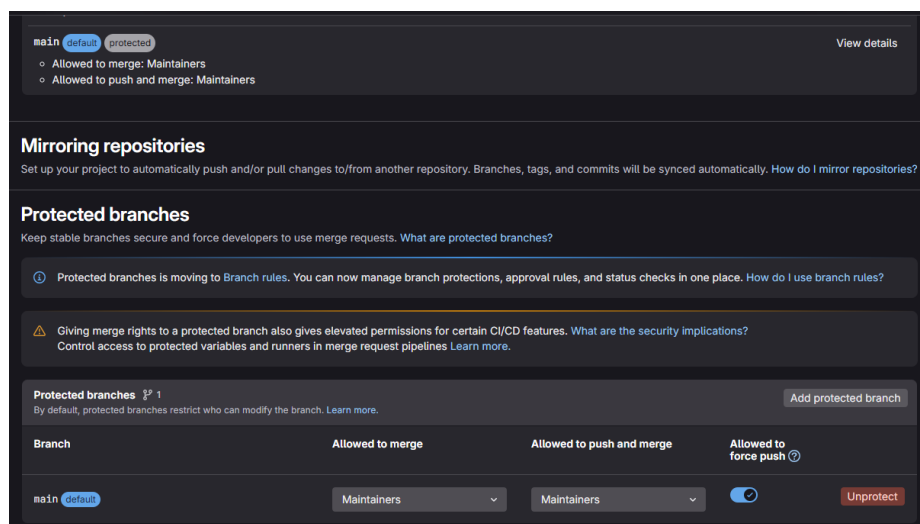


Рисунок 2 – Изменение политики ветки main

На виртуальной машине RedOS инициализируем локальный Git-репозиторий в папке с приложением и отправим код на удалённый сервер GitLab (рис. 3-4).

```
[redos@redos Crystal code (test)]$ git init -b main .
Инициализирован пустой репозиторий Git в /home/redos/Crystal code (test)/.git/
[redos@redos Crystal code (test)]$ git config --global user.email "redos@redos.com"
[redos@redos Crystal code (test)]$ git config --global user.name "redos"
[redos@redos Crystal code (test)]$ git add .
[redos@redos Crystal code (test)]$ git commit -m "Initial commit"
[main (корневой коммит) b3c72c9] Initial commit
97 files changed, 44839 insertions(+)
create mode 100644 .coverage
create mode 100644 AckWindow.py
create mode 100644 AckWindow.ui
create mode 100644 AuthWindow(mainwindow).ui
create mode 100644 AuthWindow.py
create mode 100644 AuthWindow.ui
```

Рисунок 3 – Инициализация локального репозитория

```
[redos@redos Crystal code (test)]$ git remote add origin http://gitlab.example.com/root/crystal.
git
[redos@redos Crystal code (test)]$ git branch -M main
[redos@redos Crystal code (test)]$ git push -uf origin main
warning: setting remote service path not supported by protocol
warning: setting remote service path not supported by protocol
Перечисление объектов: 108, готово.
Подсчет объектов: 100% (108/108), готово.
При сжатии изменений используется до 2 потоков
Сжатие объектов: 100% (105/105), готово.
Запись объектов: 100% (108/108), 1.83 МиБ | 2.99 МиБ/с, готово.
Total 108 (delta 22), reused 0 (delta 0), pack-reused 0 (from 0)
```

Рисунок 4 – Push-запрос в удалённый репозиторий

При запросе аутентификации укажем учётные данные администратора GitLab (логин и пароль) (рис. 5-6).

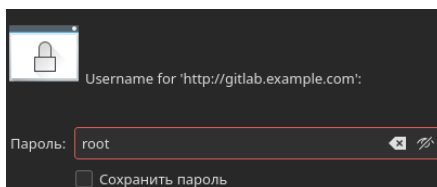


Рисунок 5 – Ввод логина

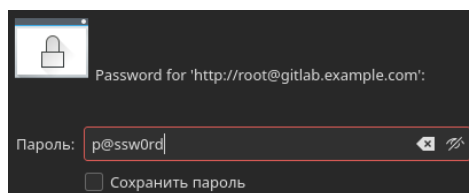


Рисунок 6 – Ввод пароля

Перенос проекта завершён успешно (рис. 7).

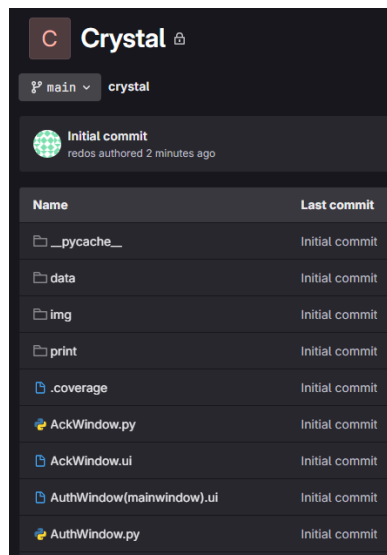


Рисунок 7 – Репозиторий Crystal в GitLab

Настроим CI/CD-пайплайн для проекта. Создадим раннер с разрешением выполнения заданий без тегов (рис. 8).

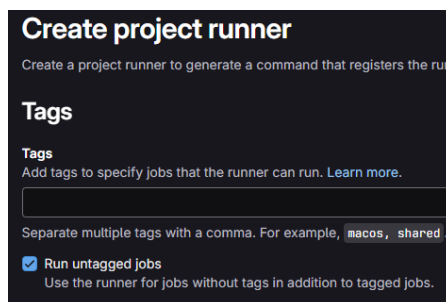


Рисунок 8 – Настройка раннера проекта

Установим gitlab-runner на виртуальной машине RedOS (рис. 9).

```
[redos@redos ~]$ curl -LJO "https://s3.dualstack.us-east-1.amazonaws.com/gitlab-runner-downloads/latest/rpm/gitlab-runner-helper-images.rpm"
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 520M 100 520M 0 0 8106k 0 0:01:05 0:01:05 --:--:-- 7904k
[redos@redos ~]$ curl -LJO "https://s3.dualstack.us-east-1.amazonaws.com/gitlab-runner-downloads/latest/rpm/gitlab-runner_amd64.rpm"
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 26.0M 100 26.0M 0 0 4182k 0 0:00:06 0:00:06 --:--:-- 5224k
[redos@redos ~]$ dnf install -y gitlab-runner-helper-images.rpm gitlab-runner_amd64.rpm
Ошибка: Эту команду нужно запускать с привилегиями суперпользователя (на большинстве систем - под именем пользователя root).
[redos@redos ~]$ sudo dnf install -y gitlab-runner-helper-images.rpm gitlab-runner_amd64.rpm
[sudo] пароль для redos:
Последняя проверка окончания срока действия метаданных: 16:38:01 назад, Пт 26 дек 2025 22:40:22.
Зависимости разрешены.
```

Рисунок 9 – Установка GitLab Runner

Выполним регистрацию и запуск раннера (рис. 10).

```
[redos@redos ~]$ sudo gitlab-runner register --url http://gitlab.example.com --token glrt--zoC
NR-bskyVVBqGUXF9am86MQpwOnoKdDozCnU6MQ8.01.1711ggt55
[sudo] пароль для redos:
Runtime platform                                arch=amd64 os=linux pid=2929 revision=cc7f92
77 version=18.7.1
Running in system-mode.

Enter the GitLab instance URL (for example, https://gitlab.com/):
[http://gitlab.example.com]:
Verifying runner... is valid                    correlation_id=01KDFZNN5466CVCCE1V86AHBN3 ru
nner=-zoCNR-bs
Enter a name for the runner. This is stored only in the local config.toml file:
[redos]: redos_runner
Enter an executor: custom, ssh, parallels, virtualbox, docker, kubernetes, docker-autoscaler, sh
ell, docker-windows, docker+machine, instance:
shell
Runner registered successfully. Feel free to start it, but if it's running already the config sh
ould be automatically reloaded!

Configuration (with the authentication token) was saved in "/etc/gitlab-runner/config.toml"
[redos@redos ~]$ sudo gitlab-runner run
Runtime platform                                arch=amd64 os=linux pid=3612 revision=cc7f92
77 version=18.7.1
Starting multi-runner from /etc/gitlab-runner/config.toml... builds=0 max_builds=0
Running in system-mode.
```

Рисунок 10 – Регистрация и запуск раннера

Раннер успешно зарегистрирован и готов к работе (рис. 11).

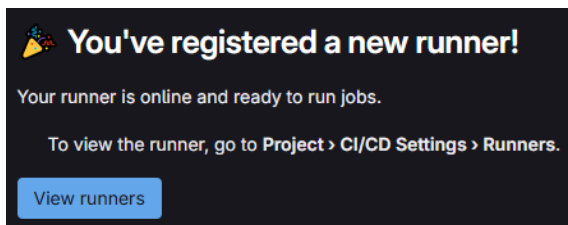


Рисунок 11 – Успешная регистрация раннера

Для сборки приложения установим компилятор gcc (рис. 12).

```
[redos@redos ~]$ sudo dnf install gcc-13.3.1-3.red80.x86_64
[sudo] пароль для redos:
Последняя проверка окончания срока действия метаданных: 4:54:07 назад, Сб 27 дек 2025 15:19:13.
Зависимости разрешены.
=====
Пакет                                Архитектура  Версия          Репозиторий    Размер
=====
Установка:
gcc                                  x86_64       13.3.1-3.red80  updates        34 М
```

Рисунок 12 – Установка gcc

Чтобы предоставить раннеру возможность отправки файлов пользователю redos, настроим беспарольный SSH-доступ от пользователя gitlab-runner. Сгенерируем SSH-ключ и добавим его в авторизованные ключи пользователя redos (рис. 13).

```
[gitlab-runner@redos ~]$ ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/gitlab-runner/.ssh/id_ed25519):
Created directory '/home/gitlab-runner/.ssh'.
Enter passphrase for '/home/gitlab-runner/.ssh/id_ed25519' (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/gitlab-runner/.ssh/id_ed25519
Your public key has been saved in /home/gitlab-runner/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:nCig6sZ84Vklb4Iyq/69j1GxKNGxFw7XqVPf8uqTY gitlab-runner@redos
The key's randomart image is:
+--[ED25519 256]--+
|      ..oo      |
|      .. .      |
|      o  o  o    |
|    + o  =o..o o  |
|o o..=.+=5.  o   |
|o. o+o+ .      |
|0 o.= .        o |
|+*.+o.      E o .|
|+==+o.      ..o o.|
+----[SHA256]-----+
[gitlab-runner@redos ~]$ ssh-copy-id redos@redos
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/gitlab-runner/.ssh/id_ed25519.pub"
The authenticity of host 'redos (fe80::e594:cf6e:1ff7:4efa%enp0s3)' can't be established.
ED25519 key fingerprint is SHA256:x9Bx6bSWk3/S7ZR6BfziYFu04hWISikom1RfXucmMJ8.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already
installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install t
```

Рисунок 13 – Настройка SSH-доступа

Создадим пайплайн, который будет автоматически выполнять тестирование, сборку и развёртывание приложения (листинг 1).

Листинг 1 – .gitlab-ci.yml

```
stages:
  - test
  - build
  - deploy
pytest:
  stage: test
  before_script:
    - echo "Установка необходимых библиотек..."
    - pip install pytest pytest-cov -r requirements.txt
  script:
    - echo "Выполнение тестирования с помощью pytest с выводом отчёта..."
    - python -m pytest --cov --junitxml=report.xml
    - echo "Тестирование завершено."
  artifacts:
    when: always
    reports:
      junit: report.xml
compile_crystal:
  stage: build
  before_script:
    - echo "Установка Nuitka..."
    - pip install Nuitka
  script:
    - echo "Выполнение сборки..."
    - python -m nuitka --follow-imports main.pyw
    - echo "Сборка завершена."
  artifacts:
```



```

paths:
  - main.bin
deploy-to-redos:
  stage: deploy
  script:
    - echo "Отправление приложения в папку пользователя RedOS..."
    - scp main.bin redos@redos:/home/redos
    - echo "Приложение успешно отправлено."

```

После сохранения файла пайплайн автоматически запускается. Все этапы успешно завершаются (рис. 14-18).

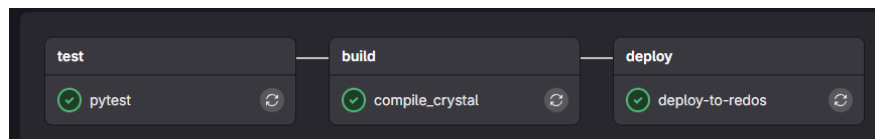


Рисунок 14 – Обзор пайплайна

```

requirements.txt (line 3)) (2.1.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/lib/python3.11/site-packages (from requests->etelemetry>=0.3.1->nipype->fritz->r requirements.t
xt (line 3)) (3.8)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/lib/python3.11/site-packages (from requests->etelemetry>=0.3.1->nipype->fritz->r requirem
ents.txt (line 3)) (2.5.0)
$ echo "Выполнение тестирования с помощью pytest с выводом отчёта..."
Выполнение тестирования с помощью pytest с выводом отчёта...
$ python -m pytest --cov --junitxml=report.xml
===== test session starts =====
platform linux -- Python 3.11.14, pytest-9.0.2, pluggy-1.6.0
rootdir: /home/gitlab-runner/builds/-zoCNR-bs/8/root/crystal
plugins: cov-7.0.0
collected 5 items
test_data.py ..... [100%]
- generated xml file: /home/gitlab-runner/builds/-zoCNR-bs/8/root/crystal/report.xml -
===== tests coverage =====
_____ coverage: platform linux, python 3.11.14-final-0 _____
Name          Stats    Miss  Cover
-----
data.py        726     185    75%
test_data.py    77        0   100%
-----
TOTAL          803     185    77%
===== 5 passed in 1.26s =====
$ echo "Тестирование завершено."

```

Рисунок 15 – Этап тестирования

```

$ echo "Установка Nuitka..."
Установка Nuitka...
$ pip install Nuitka
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: Nuitka in /home/gitlab-runner/.local/lib/python3.11/site-packages (2.8.9)
Requirement already satisfied: ordered-set>=4.1.0 in /home/gitlab-runner/.local/lib/python3.11/site-packages (from Nuitka) (4.1.0)
Requirement already satisfied: zstandard>=0.15 in /home/gitlab-runner/.local/lib/python3.11/site-packages (from Nuitka) (0.25.0)
$ echo "Выполнение сборки..."
Выполнение сборки...
$ python -m nuitka --follow-imports main.pyw
Nuitka-Options: Used command line options:
Nuitka-Options: --follow-imports main.pyw
Nuitka: Starting Python compilation with:
Nuitka: Version '2.8.9' on Python 3.11 (Flavor 'Fedora Python') commercial grade 'not installed'.
Nuitka: Completed Python level compilation and optimization.
Nuitka: Generating source code for C backend compiler.
Nuitka: Running data composer tool for optimal constant value handling.
Nuitka: Running C compilation via Scons.
Nuitka-Scons: Backend C compiler: gcc (gcc 13).
Nuitka-Scons: Backend C linking with 27 files (no progress information available for this stage).
Nuitka-Scons:WARNING: You are not using ccache, re-compilation of identical code will be slower than necessary. Use your OS package
all it.
Nuitka: Keeping build directory 'main.build'.
Nuitka: Successfully created 'main.bin'.
$ echo "Сборка завершена."
Сборка завершена.
Uploading artifacts for successful job
Uploading artifacts...
Runtime platform arch=amd64 os=linux pid=15023 revision=cc7f9277 version=18.7.1
main.bin: found 1 matching artifact files and directories
Uploading artifacts as "archive" to coordinator... 201 Created correlation_id=01KDK7CR6AK4FWXAN87Z7Z4ZWDE id=121 responseStatus=201
4_sUbjWm
Cleaning up project directory and file based variables
Job succeeded

```

Рисунок 16 – Этап сборки

```

Running with gitlab-runner 18.7.1 (cc7f9277)
  on redos_runner -zoCNR-bs, system ID: s_8eee9a2d279e
Preparing the "shell" executor
Using Shell (bash) executor...
Preparing environment
Running on redos...
Getting source from Git repository
Gitaly correlation ID: 01KDK7CVQM21KDM9BR2G6T4HNP
Fetching changes with git depth set to 20...
Перинициализирован существующий репозиторий Git в /home/gitlab-runner/builds/-zoCNR-bs/0/root/crystal/.git/
Checking out 73de8a85 as detached HEAD (ref is main)...
Удаление main.bin
Удаление main.build/
Skipping Git submodules setup
Downloading artifacts
Downloading artifacts for compile_crystal (121)...
Runtime platform arch=amd64 os=linux pid=15206 revision=cc7f9277 version=18.7.1
Downloading artifacts from coordinator... ok correlation_id=01KDK7CJNHA1KTM2S8GAB1M22Q host=gitlab.example.co
token=64_tQv-Qu
Executing "step_script" stage of the job script
$ echo "Отправление приложения в папку пользователя RedOS..."
Отправление приложения в папку пользователя RedOS...
$ scp main.bin redos@redos:/home/redos
$ echo "Приложение успешно отправлено."
Приложение успешно отправлено.
Cleaning up project directory and file based variables
Job succeeded

```

Рисунок 17 – Этап развёртывания

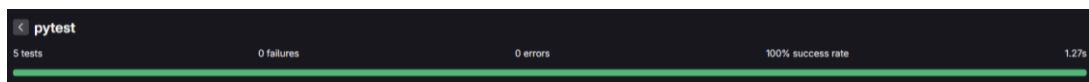


Рисунок 18 – Результаты тестов

Скомпилированный файл main.bin появился в домашней директории пользователя redos (рис. 19).

```

[redos@redos ~]$ ls
builds          gitlab-runner-helper-images.rpm  regr          Загрузки          Общедоступные
'Crystal code (test)'  main.bin                        Видео         Изображения      'Рабочий стол'
gitlab-runner_amd64.rpm  py-app                          Документы     Музыка           Шаблоны

```

Рисунок 19 – Собранное приложение в системе

Приложение успешно запускается и работает (рис. 20).

The screenshot shows the Crystal application interface with a dark blue theme. At the top, there is a header bar with the text "ИО: [Звериге] ИО пациента" and "Осмотры: []". Below this, there is a section for "Дата рождения: 15.06.2016" and "ID: []". The main area is divided into several sections: "Острота зрения" (Visual Acuity) with input fields for OD and OS, "Скисаскопия" (Sclerocopy) with input fields for OD and OS, "Очки" (Glasses) with input fields for OD and OS, and "Ша" (Steps) with input fields for OD and OS. On the right side, there is a vertical toolbar with icons for a person, a magnifying glass, a printer, and a gear. At the bottom right, there is a section for "Примечания" (Remarks) with a text area.

ИО: [Звериге] ИО пациента Осмотры: [] С расить

Дата рождения: 15.06.2016 ID: []

Острота зрения

OD: 1,00 Sph: 0,00 Cyl: 0,00 Ax: 0 = 1,00

OS: 1,00 Sph: 0,00 Cyl: 0,00 Ax: 0 = 1,00

Скисаскопия

OD [] OS []

Очки

OD Sph: 0,00 Cyl: 0,00 Ax: 0 Dpp: []

OS Sph: 0,00 Cyl: 0,00 Ax: 0 0

Ша оны

OD []

OS []

Примечания

Примечания к осмотру

Рисунок 20 – Запущенное приложение Crystal

2 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Автоматизация тестирования необходима для повышения эффективности тестового процесса за счёт сокращения временных и трудовых затрат, а также для обеспечения стабильности и повторяемости проверок;

2. Система контроля версий (VCS) — это программное обеспечение, предназначенное для управления изменениями в файлах. Оно позволяет хранить историю изменений, возвращаться к предыдущим версиям, определять автора правок и координировать совместную работу над проектом;

3. CI/CD — это методология разработки программного обеспечения, которая объединяет практики непрерывной интеграции (Continuous Integration) и непрерывного развёртывания (Continuous Deployment). Она направлена на автоматизацию процессов сборки, тестирования и поставки кода.

ВЫВОД

В ходе работы были получены практические навыки по настройке автоматизированного тестирования, сборки и развёртывания Python-приложения с использованием CI/CD-пайплайна в GitLab.